# Ders 4 Çalışma Özeti

## Ders 4 Çalışma Özeti

### Genel Konular

- Asıl ders yaklaşık 00:05'te başlar.
- Plan: ilk saat sayıcı örnekleri, ikinci saat programlanabilir lojik cihazlar.
- Sayıcı tasarımı örnekleri (geçen haftadan kalan)
  - JK flip-flop ile sayıcı: geçiş tablosu (excitation table) ile flip-flop girişlerini yazma.
  - T flip-flop ile 0, 1, 2, 4, 7 sırasında tekrar eden sayıcı.
  - Kullanılmayan durumlar (unused states) ve kendi kendini düzeltme (self-correcting) kontrolü.
- Programlanabilir lojik cihazlar (Programmable Logic Devices)
  - Ortak yapı: bir AND dizisi ve bir OR dizisi. Hangi dizinin önünün programlanabilir olduğu türü belirler.
  - ROM: sabit AND dizisi (decoder), programlanabilir OR dizisi.
  - PLA: hem AND hem OR dizisi programlanabilir.
  - PAL: programlanabilir AND dizisi, sabit OR dizisi (bu derste yetişmedi, ders 5'te anlatıldı).
  - CPLD, FPGA gibi karmaşık yapılar yalnızca adıyla anıldı.
- ROM ile kombinasyonel devre tasarımı ve ROM boyutu hesapları.
- PLA ile iki çıkışlı fonksiyon gerçekleme ve ortak terim arama.

### Hocanın Özellikle Vurguladığı Kısımlar

- JK geçiş tablosunu ezberlemeyin, fonksiyon tablosundan siz çıkarın.
  - Sadece JK flip-flop'un fonksiyon tablosunu bilmeniz yeter.
- Karnaugh haritasında küçük bir hata sonucu bozar.
  - Derste hocanın kendisi de sütun etiketini yanlış yazdı, öğrenciler düzeltti. Dikkatli olun.
- Don't care'leri 0 ya da 1 almak devrenin kullanılmayan durumlardaki davranışını değiştirir.
  - Tasarımdan sonra kullanılmayan durumların nereye gittiğini mutlaka kontrol edin.
- Kullanılmayan durumlar kendi aralarında döngü kurarsa devre kilitlenir.
  - Devre kendi kendini düzeltmiyorsa don't care seçimlerini değiştirip yeniden tasarlayın. Devre biraz büyüyebilir.
- ROM bir arama tablosudur (lookup table).
  - İçinde aritmetik yapan bir devre yoktur. Doğruluk tablosunu ROM'a yazar, adrese göre okursunuz.
- PLA tasarımında olabildiğince çok ortak terim bulun.
  - Ortak terim arttıkça gereken AND kapısı sayısı azalır.

### Kısa Tekrar Notları

- JK geçiş tablosu ($Q(t) \to Q(t+1)$: $J\,K$)
  - $0 \to 0$: $0\,X$
  - $0 \to 1$: $1\,X$
  - $1 \to 0$: $X\,1$
  - $1 \to 1$: $X\,0$
- T flip-flop: $T = 0$ ise durum korunur, $T = 1$ ise durum tümlenir. $T = Q(t) \oplus Q(t+1)$.

- Self-correcting: Kullanılmayan bir durumdan başlayan devre, birkaç saat darbesi içinde kullanılan bir duruma geçiyorsa devre kendi kendini düzeltir.
- ROM boyutu: $2^k \times n$.
  - $k$: adres girişi sayısı (devrenin giriş sayısı).
  - $n$: her satırdaki bit sayısı (devrenin çıkış sayısı).
- ROM içinde $k \to 2^k$ decoder ve $n$ tane $2^k$ girişli OR kapısı vardır.
- Sigorta (fuse) gösterimi: çarpı (×) sigorta sağlam/bağlı (intact), çarpısız sigorta atık (blown).
- Programlama: Sigortalar başta bağlıdır. İstenmeyen bağlantıların sigortası attırılır.
- PLA çıkışındaki XOR kapısı: Bir girişi 0 ise çıkış aynen geçer, 1 ise tümlenir. Böylece $F$ ya da $F'$ seçilebilir.

## Detaylı Açıklamalar

### JK flip-flop ile sayıcı tasarımı

- Durum tablosunda (state table) mevcut durum (present state) ve sonraki durum (next state) bilinir.
- D flip-flop ile tasarımda sonraki durum doğrudan D girişidir. Ayrıca giriş sütunu yazmaya gerek yoktur.
- JK flip-flop ile tasarımda her bit için $J$ ve $K$ sütunları ayrıca yazılır.
- Her satırda bitin $Q(t) \to Q(t+1)$ geçişine bakılır. Geçiş tablosundan $J\,K$ değerleri alınır.
  - Örnek: $A_2$ biti $0 \to 0$ ise $J_2K_2 = 0X$; $0 \to 1$ ise $1X$.
- Sonra her giriş ($J_2$, $K_2$, $J_1$, ...) için ayrı Karnaugh haritası çizilir ve indirgenir.
- Derste en anlamlı bit için sonuçlar:
  - $J_2 = A_1A_0'$
  - $K_2 = A_1$
  - Hoca önce $K_2 = A_0$ yazdı. Öğrenciler haritanın sütun etiketinin ($01$ yerine $10$) yanlış olduğunu gösterdi. Doğrusu $K_2 = A_1$.
- Diğer flip-flop'lar aynı yolla bulunur. Derste alıştırma olarak bırakıldı.

### T flip-flop ile 0, 1, 2, 4, 7 sayıcısı

- Sayma sırası: $0 \to 1 \to 2 \to 4 \to 7 \to 0$ (tekrarlı).
- Kullanılmayan durumlar: 3, 5, 6.
- T girişleri: bit değişiyorsa $T = 1$, değişmiyorsa $T = 0$. Kullanılmayan durumlar don't care ($X$).

| Durum ($A_2A_1A_0$) | Sonraki | $T_2$ | $T_1$ | $T_0$ |
|---|---|---|---|---|
| 000 | 001 | 0 | 0 | 1 |
| 001 | 010 | 0 | 1 | 1 |
| 010 | 100 | 1 | 1 | 0 |
| 100 | 111 | 0 | 1 | 1 |
| 111 | 000 | 1 | 1 | 1 |
| 011, 101, 110 | - | X | X | X |

- Derste bulunan indirgenmiş fonksiyonlar:
  - $T_2 = A_1$
  - $T_1 = A_2 + A_1 + A_0$ (tek 0, 000 satırında olduğu için)
  - $T_0 = A_1' + A_2$
- Bir öğrenci $T_1$ için XOR çözümünü önerdi.
  - Don't care'ler 0 alınırsa $T_1 = A_2 \oplus A_1 \oplus A_0$ olur.
  - Hoca önce "XOR çıkmıyor" dedi. Sonra öğrencinin haklı olduğunu kabul etti. Fark, don't care'lerin 0 ya da 1 alınmasından gelir.

### Kendi kendini düzeltme kontrolü (hocanın tasarımı ile)

- Durum 3 (011): $T_2T_1T_0 = 1\,1\,0$. Sonraki durum $101$ = 5. Yine kullanılmayan durum.

- Durum 5 (101): $T_2T_1T_0 = 0\,1\,1$. Sonraki durum $110$ = 6. Yine kullanılmayan durum.
- Durum 6 (110): $T_2T_1T_0 = 1\,1\,1$. Sonraki durum $001$ = 1. Kullanılan durum.
- Sonuç: $3 \to 5 \to 6 \to 1$. Devre kendi kendini düzeltir (self-correcting).
- Eğer kullanılmayan durumlar kendi aralarında döngü kursaydı (ör. $3 \to 5 \to 6 \to 3$), devre kilitlenirdi. O zaman don't care seçimlerini değiştirmek gerekirdi.

**Programlanabilir lojik cihazlar**

- Üç yapının ortak parçaları: girişte AND dizisi, çıkışta OR dizisi.
- Farkları:

| Yapı | AND dizisi | OR dizisi |
|---|---|---|
| ROM (PROM) | Sabit (decoder) | Programlanabilir |
| PAL | Programlanabilir | Sabit |
| PLA | Programlanabilir | Programlanabilir |

**ROM**

- Hocanın benzetmesi: ROM bir defter sayfası gibidir. Satır numarası adrestir. Satırda yazan bilgi çıkıştır.
- $2^k \times n$ ROM: $k$ adres girişi, $n$ veri çıkışı.
- İç yapı: $k \to 2^k$ decoder (sabit AND dizisi) ve $n$ tane OR kapısı (programlanabilir OR dizisi).
- Dizi gösterimi (array logic representation): OR kapısının tek giriş çizgisini kesen her çizgi bir giriştir. Ör. 32×8 ROM'da her OR kapısının 32 girişi vardır.
- Okuma mantığı:
  - Adres girildiğinde decoder'ın yalnızca bir çıkışı 1 olur. Diğer bütün çıkışlar 0'dır.
  - Seçilen satırda sigortası bağlı olan OR kapıları 1 üretir. Sigortası atık olanlar 0 üretir.
  - Diğer satırlardaki bağlı sigortalar çıkışı etkilemez. O satırlar 0'dır.
- ROM türleri: PROM, EPROM, EEPROM. Bazıları devre üzerinde yeniden programlanabilir.
- 8×4 ROM ($2^3 \times 4$) örneği: 3→8 decoder ve 4 tane 8 girişli OR kapısı. Adres $001$ için okunan veri $0011$.

**ROM ile 3 bitlik sayının karesi**

- Giriş $A_2A_1A_0$ (0-7). En büyük çıkış $7^2 = 49$. 49 için 6 bit gerekir ($S_5 \ldots S_0$).
  - Hoca önce 5 bit dedi, sonra 6 bit olarak düzeltti.

| Giriş | Kare | $S_5S_4S_3S_2S_1S_0$ |
|---|---|---|
| 0 | 0 | 000000 |
| 1 | 1 | 000001 |
| 2 | 4 | 000100 |
| 3 | 9 | 001001 |
| 4 | 16 | 010000 |
| 5 | 25 | 011001 |
| 6 | 36 | 100100 |
| 7 | 49 | 110001 |

- Optimizasyon:
  - $S_0 = A_0$ (her zaman). ROM'a konmaz.
  - $S_1 = 0$ (her zaman). ROM'a konmaz.
  - ROM'da yalnız $S_5S_4S_3S_2$ tutulur. Yeterli ROM: $2^3 \times 4$ (8×4).
- ROM içeriği ($S_5S_4S_3S_2$), satır 0-7: $0000, 0000, 0001, 0010, 0100, 0110, 1001, 1100$.

**ROM boyutu soruları (kitabın bölüm sonu soruları)**

- 8 bitlik toplayıcı/çıkarıcı, elde girişi (carry in) ve elde çıkışı (carry out) ile:
  - Giriş: $8 + 8 + 1$ (kontrol, topla/çıkar) $+1$ (carry in) $= 18$.
  - Çıkış: $8$ (sonuç) $+1$ (carry out) $= 9$.
  - ROM: $2^{18} \times 9$.
- İki 8 bitlik sayıyı çarpan devre:
  - Giriş 16 bit, çıkış 16 bit. ROM: $2^{16} \times 16$.
  - Bir öğrenci 15 bit yeterli mi diye sordu. Yeterli değil: $255 \times 255 = 65025 > 2^{15} = 32768$.
- 4 basamaklı BCD sayıyı ikiliye çeviren devre:
  - Giriş: $4 \times 4 = 16$ bit.
  - En büyük değer 9999. $2^{13} = 8192 < 9999 < 2^{14} = 16384$. Çıkış 14 bit.
  - ROM: $2^{16} \times 14$.

**PLA (Programmable Logic Array)**

- Girişlerde tersleyen tampon (inverting buffer) vardır. Her değişkenin hem kendisi hem tümleyeni hazırdır.
- AND kapılarının önünde ve OR kapılarının önünde programlanabilir sigortalar vardır.
- Çıkışta her fonksiyon için bir XOR kapısı vardır. XOR'un diğer girişi 0 ise $F$, 1 ise $F'$ çıkar.
- Tasarımcı yalnızca gereken AND kapısı (çarpım terimi) ve OR kapısı sayısını belirler, sigortaları ayarlar. Bu bilgi PLA programlama tablosu ile verilir.

**Örnek 1 (kitap örneği)**

- $F_1 = AB' + AC + A'BC'$
- $F_2 = (AC + BC)'$
- Ortak terim: $AC$. Toplam 4 çarpım terimi: $AB'$, $AC$, $BC$, $A'BC'$. 4 AND kapısı yeter.
- $F_1$'in XOR girişi 0, $F_2$'nin XOR girişi 1 yapılır. Böylece $F_2$ için OR çıkışının tümleyeni alınır.

**Örnek 2:** $F_1 = \Sigma(3, 5, 6, 7)$**,** $F_2 = \Sigma(1, 2, 3, 7)$

- İndirgenmiş ifadeler: $F_1 = AB + AC + BC$, $F_2 = A'B + A'C + BC$.
  - Ortak terim yalnız $BC$. Toplam 5 AND kapısı.
- Çıkıştaki XOR sayesinde bir fonksiyonun tümleyeni de kullanılabilir. Ortak terimleri artıran seçenek aranır.
  - $F_1' = \Sigma(0, 1, 2, 4)$ ile $F_2$'nin indirgenmemiş biçimi kullanılırsa iki ortak terim çıkar. Toplam 4 AND kapısı.
  - Bir çözüm (ders sonrası Python ile doğrulandı): $F_1' = B'C' + A'BC' + A'B'C$, $F_2 = BC + A'BC' + A'B'C$. Ortak terimler: $A'BC'$ ve $A'B'C$.
- Ayrı ayrı indirgenmiş haller her zaman en az AND kapısını vermez. İndirgenmemiş hallere de bakın.
- Hoca bir ek PLA örneği daha yazıp çözümünü size bıraktı. Çözüm yolunu ders 5'in başında özetledi.

**Ders sonrası notlar**

- $K_2$ **hatası:** Hoca $K_2$ haritasında sütun etiketlerini $00, 01, 11, 01$ yazdı ve sonucu $A_0$ buldu. Doğru sıra Gray kodudur: $00, 01, 11, 10$. Doğru sonuç $K_2 = A_1$. Ders içinde öğrenciler düzeltti.
- **XOR çözümü:** Hoca önce "$T_1$ için XOR çıkmıyor" dedi. Don't care'ler 0 alındığında $T_1$ yalnız 1, 2, 4, 7 durumlarında 1 olur. Bu durumlarda tek sayıda 1 vardır. Yani $T_1 = A_2 \oplus A_1 \oplus A_0$ doğrudur. Hoca bunu sonra kabul etti.
  - Bu seçimle 3, 5, 6 durumlarında $T_1 = 0$ olur. Kendi kendini düzeltme kontrolü yeniden yapılmalıdır.
- **Kare ROM'u:** 6 satırın değeri derste önce $1010$ diye söylendi, sonra $1001$ olarak düzeltildi. Doğrusu $36 = 100100_2$, yani $S_5S_4S_3S_2 = 1001$.

- **Çarpıcı ROM'u:** Hoca "kontrol edin" dedi. 15 bit yetmez; çarpım en fazla $65025$ olur ve 16 bit gerekir. $2^{16} \times 16$ doğrudur.
- **Örnek 2:** Kayıtta tahtadaki ifadeler tam okunmuyor. Yukarıdaki 4 terimli çözüm tüm seçenekler denenerek bulundu. $F_1$ ve $F_2$ doğrudan alınırsa en az 5, $F_1'$ ile $F_2$ (ya da $F_1$ ile $F_2'$) alınırsa en az 4 terim gerekir.
- **Kendi kendini düzeltme:** Hoca "sistem unused state'lere geçebilsin istiyoruz" dedi. Amaç tersidir: devre kullanılmayan durumdan kullanılan (used) duruma geçebilmelidir.
- **Kare ROM'u, 2. satır:** Hoca "ikinci satırda $A_2$ sadece kapalı" dedi. Satır 2'nin içeriği $0001$'dir. Yani yalnız $S_2$ çıkışının sigortası bağlıdır. $A_2$ bir adres girişidir.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.